ITM SKILLS UNIVERSITY

School of Future Tech

Case Study Number 170 - Report

on

Online Shopping Cart System (C++)

by

Abdeali Makda

150096725096

Cohort: Larry Page

Academic Year 2025–2029

Index

1. Introduction to the Case Study

In today's technology-driven world, object-oriented programming (OOP) serves as the backbone of most real-world software systems. This case study presents the design and implementation of an Online Shopping Cart System built entirely in C++ — a console-based application that simulates the core workflows of a modern e-commerce platform. The project was developed to demonstrate the practical application of fundamental OOP concepts in a domain that is immediately recognisable and relatable: online shopping.

The system allows users to browse a pre-loaded product catalog containing 10 items across two categories — Electronics and Accessories — add or remove products from a cart, view itemised bills with GST applied, apply discount coupons, and save completed orders to a file for persistent order history. Every interaction is driven by a numbered menu interface, making the program straightforward to operate while being architecturally rich under the hood.

This project was implemented as a single-file C++ program (main.cpp) to keep the codebase accessible and self-contained while still showcasing a comprehensive class hierarchy, runtime polymorphism through virtual functions, dynamic memory allocation, and file-based data persistence.

Key Features at a Glance

- Product Catalog — 10 pre-loaded products across Electronics and Accessories categories.
- Add to Cart — Add any product by ID; duplicate entries automatically increment quantity.
- Remove from Cart — Decrease quantity by one; item is fully removed when quantity reaches zero.
- View Cart — Formatted table with base price, GST (18%), and subtotal per item.
- Checkout — Computes GST, applies an optional discount percentage, and shows the final payable amount.
- Order History — Every completed order is appended to orders.txt and can be reviewed at any time.
- Menu-Driven Interface — Simple numbered menu loop for smooth and intuitive navigation.

2. Problem Statement / Case Background (Abstract)

Background

Academic programming assignments often focus on isolated concepts — a linked list here, a sorting algorithm there — without providing the broader context of how these concepts combine to form a complete, functional system. Students learning C++ OOP frequently struggle to see how abstract base classes, virtual dispatch, and dynamic memory allocation work together in practice.

At the same time, e-commerce is one of the most ubiquitous application domains in modern software development. Shopping carts, product catalogs, billing engines, and order histories are components that every software professional will encounter in some form. A C++ simulation of these components, implemented with proper OOP design, provides a compelling context for learning that connects directly to industry-relevant patterns.

The core challenge addressed by this case study is: how can a console-based C++ program model the behaviour of a real-world shopping system while simultaneously demonstrating the full spectrum of OOP principles — abstraction, inheritance, polymorphism, encapsulation, and file handling — in a clean and maintainable architecture?

Abstract

This case study presents the design and complete implementation of a console-based Online Shopping Cart System in C++. The system models an online store with a polymorphic product hierarchy: an abstract base class Product with two derived classes, Electronics and Accessory, each implementing virtual functions for GST calculation, category retrieval, and product display.

A CartItem class associates a polymorphic product pointer with a quantity, and a ShoppingCart class manages all cart operations including adding, removing, and viewing items, as well as checkout with optional discount application. Completed orders are persisted to a file (orders.txt) using C++ file streams, and past orders can be retrieved and displayed at runtime.

The program operates entirely through a menu-driven interface and is implemented as a single-file project (main.cpp), making it immediately compilable on any standard C++ environment. It demonstrates eight distinct OOP and C++ concepts: classes and objects, abstraction, inheritance, polymorphism, encapsulation, virtual destructors, dynamic memory allocation, and file handling.

3. Case Study Design

The system is designed around a clean object-oriented class hierarchy with clearly separated responsibilities. Each class has a single, well-defined role, and interactions between classes are mediated through public interfaces.

Class Architecture

Class / Component	Role
Product (Abstract Base)	Blueprint for all products. Stores id, name, price as public members. Declares pure virtual functions getGST(), getCategory(), display().
Electronics (Derived)	Represents electronic products. Implements all three pure virtual functions. GST = 18% of base price.
Accessory (Derived)	Represents accessory products. Implements all three pure virtual functions. GST = 18% of base price.
CartItem	Pairs a Product* (polymorphic pointer) with a quantity. Computes line-item subtotal including GST.
ShoppingCart	Manages a fixed-size array of CartItems. Handles add, remove, view, and checkout operations.
main() + free functions	Initialises the product catalog as an array of Product* pointers. Runs the menu loop. Handles file display.

Application Flow

1. Browse Catalog — User selects option 1. showCatalog() calls display() on each Product* pointer via virtual dispatch.
2. Add Product — User enters a Product ID. ShoppingCart::addProduct() checks for duplicates via findIndex(); if found, increments quantity; otherwise creates a new CartItem.
3. View Cart — ShoppingCart::viewCart() renders a formatted table with per-item base price, GST, subtotal, and a grand total row.
4. Remove Product — ShoppingCart::removeProduct() decrements quantity by 1; when it hits zero, the array is compacted by left-shifting all subsequent elements.
5. Checkout — ShoppingCart::checkout() displays the full cart, prompts for an optional discount percentage, computes the final bill, prints a formatted summary, and calls saveOrder() to append the order to orders.txt.
6. View Order History — displaySavedOrders() opens orders.txt with ifstream and prints all previously saved orders line by line.

Data Model

All persistent data is written to a single text file, orders.txt, using pipe-separated values. Each order block contains one line per CartItem (ID | name | category | qty | subtotal), followed by five summary lines (BaseTotal, GSTTotal, Discount%, DiscountAmt, TotalPayable), and a --- separator between orders. The in-memory catalog is a fixed-size array of 10 Product* pointers pointing to heap-allocated Electronics and Accessory objects.

4. Methods, Algorithms & Technology Applied

4.1 Abstract Base Class and Pure Virtual Functions

The Product class is declared abstract by including three pure virtual functions. This design choice serves two purposes: it enforces a contract that every derived class must implement getGST(), getCategory(), and display(), and it enables the catalog to be stored as an array of Product* base class pointers while still invoking the correct derived class behaviour at runtime. No Product object can be instantiated directly — only Electronics and Accessory objects can.

4.2 Runtime Polymorphism via Virtual Dispatch

The product catalog is declared as Product* catalog[N]. When showCatalog() iterates this array and calls catalog[i]->display(), the C++ virtual dispatch mechanism resolves the call to Electronics::display() or Accessory::display() depending on the actual type of the object at runtime. The same applies to getGST() calls inside CartItem::getSubtotal() — the correct GST formula is applied for each product type without the calling code needing to know which type it is dealing with.

4.3 Cart Management Algorithm

ShoppingCart maintains a CartItem items[MAX] array with a count tracker. The findIndex() helper performs a linear scan to check whether a given product ID already exists in the cart. The addProduct() function uses findIndex() to decide between incrementing an existing item's quantity and creating a new CartItem. The removeProduct() function similarly locates the item, decrements its quantity, and — if the quantity reaches zero — compacts the array by shifting all subsequent elements one position left, eliminating the need for a separate deletion flag.

4.4 GST and Billing Calculation

Component	Formula	Example (Rs. 10,000 base)
GST per unit	price × 0.18	Rs. 1,800
Subtotal per item	(price + GST) × qty	Rs. 11,800 (qty = 1)
Grand subtotal	Sum of all subtotals	—
Discount amount	subtotal × (disc% / 100)	Rs. 590 (5% disc)
Total payable	subtotal - discountAmount	Rs. 11,210

4.5 File Handling for Order Persistence

Completed orders are saved by opening orders.txt with ofstream in ios::app (append) mode. This ensures that every new order is added at the end of the file without overwriting previous entries. Each call to saveOrder() writes all cart items as pipe-separated lines followed by the billing summary and a --- separator. Order history is retrieved by opening the same file with ifstream and reading it line by line using getline(), printing each line to the console.

4.6 Technology Stack

Component	Technology / Approach
Language	C++ (C++11 standard)
OOP Paradigm	Abstract classes, inheritance, runtime polymorphism, encapsulation
Memory Management	Manual dynamic allocation with new / delete
File I/O	C++ Standard Library: ofstream (ios::app), ifstream, getline()
Formatted Output	printf() with format specifiers for aligned table rendering
User Input	cin with switch-case menu dispatch
Build Environment	Any C++ compiler (g++, clang++, MSVC); single-file compilation
Data Persistence	Text file (orders.txt) with pipe-separated values and order separators

5. Implementation Details

5.1 Project Structure

File	Description
main.cpp	Complete source code — all class definitions, member functions, and the main() entry point in a single file.
orders.txt	Auto-generated at first checkout. Stores all completed orders in pipe-separated format. Appended to on each checkout.
README.md	Project documentation covering features, class overview, OOP concepts, step-by-step usage guide, and GST calculation table.

5.2 Class and Member Function Reference

Class	Key Members	Responsibility
Product	virtual getGST(), virtual getCategory(), virtual display(), virtual ~Product()	Abstract blueprint; common product attributes and contract.
Electronics	getGST(), getCategory(), display()	Electronic product; GST = price × 0.18; category label = "Electronics".
Accessory	getGST(), getCategory(), display()	Accessory product; GST = price × 0.18; category label = "Accessory".
CartItem	getSubtotal(), addOne(), removeOne()	Line item pairing a Product* with a quantity; computes (price+GST)×qty.
ShoppingCart	addProduct(), removeProduct(), viewCart(), checkout(), saveOrder(), isEmpty()	Full cart management: add, remove, display, bill, and persist orders.

5.3 Key Implementation Decisions

Single-File Architecture: The entire program — five classes and all free functions — is contained within main.cpp. This makes the project immediately compilable with a single command (g++ main.cpp -o shopping_cart) and keeps the codebase easily navigable for learning purposes. Detailed block comments at the top of each class explain design decisions inline.

char[] Instead of std::string: The Product class stores the name as char name[40] rather than std::string. This choice aligns with C-style character array handling and keeps the Product class free of Standard Library dependencies beyond the basics, which is appropriate for a systems-oriented C++ learning project.

Static const MAX in ShoppingCart: The cart capacity is defined as static const int MAX = 20 — a compile-time constant scoped to the ShoppingCart class. This is preferable to a raw magic number and avoids the overhead of dynamic array resizing while keeping the capacity declaration close to the class that uses it.

Public Members for Simplicity: In the simplified implementation, the data members id, name, and price in Product, and product and qty in CartItem, are declared public rather than private. This removes the need for getter functions.

Virtual Destructor on Product: Product declares virtual ~Product() {} to ensure that when delete catalog[i] is called at program exit — where catalog[i] is a Product* pointing to a heap-allocated Electronics or Accessory — the correct derived class destructor is invoked first, followed by the base class destructor. Without this, deleting through a base pointer without a virtual destructor is undefined behaviour in C++.

Array Compaction on Remove: When a CartItem's quantity reaches zero after a removeOne() call, removeProduct() shifts all subsequent elements in items[] one position left rather than leaving a gap or using tombstone flags. This keeps the count accurate and the array compact, avoiding the need for skip-logic in viewCart() and checkout().

5.4 Product Catalog

ID	Product Name	Category	Base Price (Rs.)
1	Laptop	Electronics	55,000.00
2	Smartphone	Electronics	22,000.00
7	Monitor 24 inch	Electronics	18,000.00
8	Webcam HD	Electronics	3,000.00
9	SSD 1TB	Electronics	7,500.00
3	Wireless Headphones	Accessory	3,500.00
4	USB-C Hub	Accessory	1,200.00
5	Mechanical Keyboard	Accessory	4,500.00
6	Gaming Mouse	Accessory	2,200.00
10	Smart Watch	Accessory	9,999.00

6. Results and Conclusion

Findings

The following outcomes were observed across development and testing of the Online Shopping Cart System:

- Polymorphism: Virtual dispatch correctly routes getGST() and display() calls to the appropriate derived class at runtime. The catalog array of Product* pointers behaves seamlessly — adding new product types would require no changes to ShoppingCart or the menu logic, only a new derived class.
- Cart Accuracy: The addProduct() duplicate-detection logic (via findIndex()) consistently increments quantity for repeated additions rather than creating duplicate rows. The array compaction in removeProduct() keeps the item list clean and the total calculations accurate across all test scenarios.
- GST and Billing: GST at 18% is calculated correctly per unit and aggregated at checkout. Discount input is validated to the range 0–100; invalid inputs fall back to 0% with a notification. The final bill breakdown — base total, GST, discount, and total payable — matches expected values in all tested cases.

- File Persistence: Orders are correctly appended to orders.txt across multiple checkout sessions without overwriting prior entries. The --- separator between orders makes the file human-readable and allows the display function to present all orders in sequence.
- Memory Management: All 10 heap-allocated Product objects are deleted in the cleanup loop at program exit. The virtual destructor chain ensures that the correct destructors are called for both Electronics and Accessory objects when deleted via Product* pointers.
- Menu Interface: The switch-case menu loop handles all six options correctly. Invalid choices print an error and re-display the menu. Edge cases — such as attempting to checkout or remove from an empty cart — are caught and reported gracefully.

Limitations

- No persistent cart state: The shopping cart exists only in memory for the duration of the program. If the user exits without checking out, all cart contents are lost. A production implementation would serialise the cart to a file on exit and reload it on start.
- Fixed-size array: The cart is capped at 20 unique items (MAX = 20) and the catalog at 10 products (N = 10). These limits are appropriate for a demonstration but would need to be replaced with dynamic containers (std::vector) in a real application.
- Single GST rate: Both Electronics and Accessory categories use 18% GST. In the actual Indian GST framework, different product categories attract different tax slabs. Extending the system to support varied rates would require only modifying the getGST() implementations in each derived class.
- No user authentication or multi-user support: The program models a single-session, single-user scenario. A full-featured system would require user accounts and session management.

Conclusion

The Online Shopping Cart System successfully demonstrates all eight targeted C++ and OOP concepts — classes and objects, abstraction, inheritance, runtime polymorphism, encapsulation, virtual destructors, dynamic memory allocation, and file handling — within the context of a coherent, functional application.

The project illustrates that abstract class hierarchies and polymorphism are not merely academic constructs; they are practical design tools that allow new product types to be added to the system without modifying existing cart or checkout logic. The clean separation between the Product hierarchy, CartItem, and ShoppingCart mirrors the single-responsibility and open-closed principles that underpin professional software design.

Future extensions to this system could include replacing the fixed-size arrays with std::vector for dynamic resizing, adding a user authentication layer, implementing category-specific GST rates,

supporting multiple discount coupon codes, and porting the interface to a graphical or web-based frontend while reusing the core C++ business logic through a REST API wrapper.

7. References

1. Stroustrup, B. (2013). The C++ Programming Language (4th ed.). Addison-Wesley. — Comprehensive reference for C++ language features, OOP design, and virtual function mechanics.
2. Lippman, S. B., Lajoie, J., & Moo, B. E. (2012). C++ Primer (5th ed.). Addison-Wesley. — Detailed treatment of inheritance, polymorphism, dynamic binding, and file I/O in C++.
3. cppreference.com — C++ Standard Library reference for std::ofstream, std::ifstream, ios::app mode, getline(), and printf() format specifiers. https://en.cppreference.com
4. GeeksforGeeks — Articles on abstract classes in C++, virtual functions and vtable, pure virtual functions, and dynamic memory management with new/delete. https://www.geeksforgeeks.org/cpp-polymorphism/
5. Government of India, GST Council — GST rate schedule for electronics and accessories. Reference for the 18% GST slab applied in the billing module. https://www.gst.gov.in
6. ISO/IEC 14882:2011 (C++11 Standard) — Specification for C++ language features used in this project including virtual destructors, override keyword, and static data members.

Declaration

I hereby declare that this case study report on the Online Shopping Cart System (C++) is an original work completed by me. All external references and resources have been duly cited. The application, its code, and the analysis presented in this report are my own work unless otherwise stated.

Abdeali Makda
Roll No: 150096725096
Cohort: Larry Page
School of Future Tech, ITM Skills University